

Forever — iOS Weight Tracking App: Implementation Plan

Context

A demo-only iOS app for tracking weight changes over time across multiple family member profiles. One person operates the app and tracks weight on behalf of others (e.g. the whole family from one phone). The app has a distinctive customisable kitchen home screen as its visual identity. Delivery timeframe is short. The engineer has React Native experience but limited mobile experience more broadly.

Recommended Tech Stack

Concern	Choice	Reason
Framework	React Native with Expo SDK 51	Engineer already knows React Native; Expo removes iOS build complexity; Expo Go enables instant demo sharing via QR code
Language	TypeScript	Type safety; familiar if they know JS/TS
Navigation	Expo Router (file-based)	Built into Expo; similar to Next.js file routing; handles auth redirect natively
State	Zustand (<code>zustand ^4.4.0</code>)	Minimal boilerplate; no Provider tree needed; familiar store pattern
Database	expo-sqlite + Drizzle ORM (<code>drizzle-orm ^0.29.0</code>)	expo-sqlite is built into Expo; Drizzle gives type-safe queries similar to Drift/Prisma; reactive queries via subscriptions
Auth	expo-secure-store + <code>crypto-js</code>	iOS Keychain via Expo; no backend needed; offline-safe demo
SVG	react-native-svg (<code>react-native-svg ^14.1.0</code>)	Most widely used SVG library in RN ecosystem; Expo managed workflow supports it
Charts	Victory Native XL (<code>victory-native ^41.x</code>)	Purpose-built for RN; line charts with good touch interaction; Expo-compatible
Deployment	Expo Go (demo day) → EAS Build + TestFlight (sharing)	Expo Go = instant QR-code demo, no provisioning; EAS for wider sharing
IDE	VS Code + Expo Tools / React Native Tools extension	Standard setup

Why React Native over Flutter (given this engineer):

- Existing RN experience eliminates the Dart/Flutter learning curve entirely
- Expo Go allows zero-friction demo sharing — no Xcode, no device provisioning needed
- JS/TS ecosystem the engineer already knows
- The minor performance difference (JS bridge vs compiled Dart) is irrelevant for a demo-scale app

Project Scaffold

```
npx create-expo-app@latest forever --template blank-typescript
cd forever
npx expo install expo-sqlite expo-secure-store expo-router react-native-svg zustand drizzle-orm drizzle-kit crypto-js victory-native react-native-reanimated
```

Directory layout (target):

```
app/
(auth)/
  _layout.tsx      # Auth stack navigator
  login.tsx
  create-account.tsx
(tabs)/
  _layout.tsx      # Tab navigator (4 tabs)
  index.tsx        # Home screen (kitchen scene)
  measurements.tsx # Timeline + graph tabs
  profiles.tsx     # Profile list
  collaborative.tsx

src/
db/
  schema.ts        # Drizzle table definitions
  database.ts       # expo-sqlite connection + Drizzle instance
  migrations/      # Auto-generated by drizzle-kit
services/
  auth.service.ts
  profile.service.ts
  measurement.service.ts
utils/
  weight.ts         # Unit conversion: kg ↔ lbs ↔ stones/lbs
store/
  auth.store.ts     # Zustand auth state
  activeProfile.store.ts
components/
  kitchen/
    KitchenScene.tsx
    KitchenTable.tsx
    KitchenBowl.tsx
    KitchenPlantpot.tsx
    KitchenWall.tsx
    KitchenAvatar.tsx # avatar standing in scene; sub-components: AvatarBody, AvatarFace, AvatarHair
  measurement/
    MeasurementTile.tsx
    MilestoneBadge.tsx
  shared/
    AvatarCustomiser.tsx # gender/width/skin/hair selector with live preview
    WeightInput.tsx

assets/
svg/                # 16 SVG files (4 elements × 4 variants)
```

Data Model (Drizzle + expo-sqlite)

All data is local to the device. No backend or network required.

users table

```
id: text primaryKey,      // UUID
displayName: text notNull,
createdAt: integer notNull // Unix ms
```

profiles table

```

id: text primaryKey,
userId: text notNull references(users.id),
name: text notNull,
avatarGender: text default('male'), // 'male' | 'female'
avatarWidth: integer default(0), // 0-3 (slim → larger)
avatarSkinTone: integer default(0), // 0-3
avatarHairStyle: integer default(0), // 0-3
tableVariant: integer default(0), // 0-3
bowlVariant: integer default(0),
plantpotVariant: integer default(0),
wallVariant: integer default(0),
weightUnit: text default('kg'), // 'kg' | 'lbs' | 'st' — per profile
createdAt: integer notNull

```

`measurements` table

```

id: text primaryKey,
profileId: text notNull references(profiles.id),
weightKg: real notNull,
measuredAt: integer notNull, // Unix ms (noon UTC for date-only values)
isMilestone: integer default(0), // 0/1 boolean
milestoneName: text, // nullable
notes: text, // nullable
createdAt: integer notNull

```

Indexes: `(profileId, measuredAt)` and `(profileId, isMilestone)` on measurements.

Cascade deletes: profile delete → cascade to measurements.

Weights stored in kg only in DB always. The `weightUnit` on the profile controls display and input only — conversion handled entirely in `src/utils/weight.ts`.

Migration strategy for demo: `drizzle-kit push` to apply schema changes; no backwards-compat needed for demo data.

Weight unit conversion utility (`src/utils/weight.ts`):

- `toKg(value, unit, lbsPart?)` — converts user input to kg for storage
- `fromKg(kg, unit) → { primary, secondary? }` — converts stored kg to display unit (stones returns `{ primary: stones, secondary: lbs }`)
- `formatWeight(kg, unit) → string: "72.5 kg" / "159.8 lbs" / "11 st 4 lbs"`

Sub-Plans per Requirement

R1 — Simple Deployment

- Demo day: `npx expo start` → share QR code → anyone opens in Expo Go on their iPhone. No provisioning, no TestFlight, no Xcode required.
- Beyond demo day: `eas build --platform ios --profile preview` → TestFlight distribution. Requires free Apple Developer account (USD \$0 for TestFlight to invited testers).
- iOS only: Set `"platforms": ["ios"]` in `app.json`; install `expo-build-properties` to set iOS deployment target (e.g. 16.0).

R2 — Login

Auth flow:

1. App starts → check `expo-secure-store` for key `"forever_credentials"`. If absent → `/(auth)/create-account`.
2. `create-account`: display name + password + confirm → `AuthService.createAccount()`:
 - Generate UUID for `userId`
 - SHA-256 hash password + random salt (via `crypto-js`)
 - Store `{userId, displayName, passwordHash, salt}` as JSON in Secure Store
3. `login`: read Secure Store → hash input + stored salt → compare → on match set `auth.store` with `userId`

4. Zustand auth store: `{ userId: string | null, displayName: string | null, login(), logout() }`
5. Expo Router auth redirect: Root `_layout.tsx` reads auth store; if `userId === null` redirects to `/(auth)/login`
6. No network calls, works fully offline

Key files: `src/services/auth.service.ts` , `src/store/auth.store.ts` , `app/(auth)/login.tsx` , `app/(auth)/create-account.tsx`

R3 — Profiles

- `app/(tabs)/profiles.tsx` : lists all profiles for current `userId` via Drizzle `select().from(profiles).where(eq(profiles.userId, userId))`
- "Add Profile" FAB → modal `CreateProfileScreen` : name field + weight unit picker + avatar customiser + default kitchen config → INSERT into `profiles`
- Tap profile row → `EditProfileScreen` : pre-populated form including unit picker and avatar customiser; DELETE with confirmation dialog (Drizzle cascade removes measurements)
- Profile list row: shows the avatar rendered at small scale using `KitchenAvatar` component (same SVG, smaller `viewBox` crop)

Key files: `app/(tabs)/profiles.tsx` , `src/services/profile.service.ts`

R4 — Kitchen Home Screen

Asset strategy: 16 SVG files in `assets/svg/` — one per variant, named `table_oak.svg` , `table_white.svg` , ... `wall_terracotta.svg` .

Scene composition: `KitchenScene` is a functional component using an `View` with `position: 'relative'` and fixed `aspectRatio: 4/3` . Five `SvgUri` or `SvgXml` layers from `react-native-svg` stacked with `position: 'absolute'` :

- Layer 0 (bottom): Wall — full background
- Layer 1: Table — centred, lower half
- Layer 2: Bowl — centred on table surface
- Layer 3: Plantpot — right side
- Layer 4: Avatar — standing in front of the table, left-of-centre

Positions use `width/height` as percentages of parent so the scene scales to any screen.

Variant mapping:

```
// src/components/kitchen/variants.ts
const tableAssets = {
  0: require('../../assets/svg/table_oak.svg'),
  1: require('../../assets/svg/table_white.svg'),
  2: require('../../assets/svg/table_black.svg'),
  3: require('../../assets/svg/table_grey.svg'),
} as const;
// same pattern for bowl, plantpot, wall
```

Customisation screen: `CustomiseKitchenScreen` — 4 horizontal `ScrollView` rows, one per element, each showing 4 thumbnail `SvgUri` tiles (64×64). Tap a tile → `profileService.updateKitchenConfig()` → Zustand `activeProfile` updates → home screen re-renders.

SVG placeholder strategy: If SVG artwork is not ready, replace `SvgUri` with `<View style={{backgroundColor: variantColor}} />` — the `KitchenScene` component API is unchanged when real SVGs arrive.

Key files: `src/components/kitchen/KitchenScene.tsx` , `src/components/kitchen/variants.ts` , `src/components/kitchen/KitchenAvatar.tsx` , `app/(tabs)/index.tsx` , `src/screens/CustomiseKitchenScreen.tsx`

R5 — Profile Switching

- `activeProfile.store.ts` (Zustand): `{ profile: Profile | null, setProfile(p), clearProfile() }`
- On login: auto-set to first profile alphabetically (Drizzle query `ORDER BY name ASC LIMIT 1`)
- Persist last-active profileId to `AsyncStorage` (`@react-native-async-storage/async-storage`); restore on app restart

- Profile switcher: A `Pressable` in the tab bar header (or a persistent floating element) showing current profile name + chevron. Tap → `BottomSheetModal` from `@gorhom/bottom-sheet` listing all profiles. Tap row → `setProfile()` → all screens re-render via Zustand subscription
- All screens read `useActiveProfileStore(s => s.profile)` — updates flow automatically

Key files: `src/store/activeProfile.store.ts` , shared header component in `app/(tabs)/_layout.tsx`

R6 — Measurements (Input & Storage)

- `AddMeasurementScreen` (modal, pushed over any tab) reads the active profile's `weightUnit` and renders the appropriate input:
 - kg or lbs: single `TextInput` , `keyboardType="decimal-pad"` , unit suffix label ("kg" or "lbs")
 - stones: two `TextInput` fields side by side — `[11] st [4] lbs` — both `keyboardType="number-pad"`
 - Date row: shows selected date formatted as `dd MMM yyyy` ; tap opens `DateTimePickerModal` from `react-native-modal-datetime-picker` (`min: new Date(1900, 0, 1)` , `max: today`)
 - Optional notes `TextInput`
 - Milestone `Switch` — when on, animates in milestone name `TextInput`
 - Save → call `toKg(value, unit, lbsPart?)` → INSERT `weightKg` into `measurements`
- Validation: converted kg value > 0 and < 500; date not in future
- Unit preference set on `EditProfileScreen` (dropdown: kg / lbs / stones)
- Historical entry: the date picker supports any date range — no special handling needed
- Editing: long-press any `MeasurementTile` → same modal pre-populated (UPDATE query)

Key files: `src/screens/AddMeasurementScreen.tsx` , `src/services/measurement.service.ts`

R7 — Measurement Display (Timeline + Graph)

Timeline:

- `FlatList` driven by Drizzle `select().from(measurements).where(eq(measurements.profileId, id)).orderBy(desc(measurements.measuredAt))`
- Each row: `MeasurementTile` — date, `formatWeight(weightKg, profile.weightUnit)` , `MilestoneBadge` if applicable
- Filter control at top: segmented control "All" / "Milestones" — adds `.where(eq(measurements.isMilestone, 1))` to the query
- Long-press → edit/delete modal

Graph:

- Same query sorted `ASC` by `measuredAt`
- Map to `[{ x: Date, y: number }]` array for Victory Native XL
- `VictoryLine` with `VictoryScatter` for data points; milestone points use custom `dataComponent` (amber star symbol)
- `VictoryAxis` bottom: date formatted `MMM 'yy` ; left axis: weight in the profile's unit
 - kg → y values in kg, axis labels in kg
 - lbs → y values in lbs, axis labels in lbs
 - stones → y values in lbs (decimal stones is meaningless to read); tooltip shows converted `"11 st 4 lbs"`
- `VictoryVoronoiContainer` for touch tooltip showing date + `formatWeight(kg, unit)`

Container: `app/(tabs)/measurements.tsx` hosts both views in a top `Tab` component (React Navigation `createMaterialTopTabNavigator` or simple custom tab bar with two `Pressable` headers).

Key files: `app/(tabs)/measurements.tsx` , `src/components/measurement/` , chart config in dedicated file

R8 — Milestones

- Milestone is a property of a `Measurement` row (`isMilestone: 0|1` , `milestoneName: text`)
- Added during measurement creation (see R6) or retroactively via edit modal

- **MilestoneBadge** : small amber-background **View** with star icon (`@expo/vector-icons/MaterialIcons` star) + milestone name text; inline in **MeasurementTile**
- Timeline filter: segment "Milestones" adds `WHERE isMilestone = 1` clause
- Graph: milestone data points use a distinct larger amber scatter marker (`VictoryScatter` with custom `dataComponent`)

R9 — Collaborative View

- `app/(tabs)/collaborative.tsx` : horizontally scrollable **ScrollView** or **FlatList horizontal** — one **ProfileSummaryCard** per profile
- Each card:
 - Profile name + avatar emoji
 - Latest weight: `SELECT ... ORDER BY measuredAt DESC LIMIT 1` — displayed via `formatWeight(weightKg, profile.weightUnit)`
 - Most recent milestone label: `SELECT ... WHERE isMilestone=1 ORDER BY measuredAt DESC LIMIT 1`
 - Mini kitchen scene: **KitchenScene** wrapped in **View** scaled down via `transform: [{ scale: 0.4 }]` + `overflow: 'hidden'`
- Data loaded via a custom hook `useCollaborativeData(userId)` that runs all profile + measurement queries and returns a combined array
- No dependency on `activeProfileStore` — always shows all profiles

Key files: `app/(tabs)/collaborative.tsx` , `src/hooks/useCollaborativeData.ts`

R11 — Customisable Profile Avatar (standing in kitchen scene)

Avatar options:

- Gender: male / female (2 options) — affects body silhouette and base hair shape
- Width: 4 options — slim, regular, athletic, larger — controls body proportions
- Skin tone: 4 options — light, medium, olive, dark — fills face, neck, hands
- Hair style: 4 options — short, long, curly, shaved — shape differs per style; gender affects the silhouette of each

Total combinations: $2 \times 4 \times 4 \times 4 = 128$ unique avatars from one programmatic SVG component.

Component structure (`src/components/kitchen/KitchenAvatar.tsx`):

The avatar is a single `<G>` (SVG group) containing three stacked sub-components, all drawn programmatically using `react-native-svg` primitives — no external SVG files needed:

```
KitchenAvatar
├── AvatarBody — torso + legs; shape driven by width variant + gender
├── AvatarFace — head circle + neck; fill colour driven by skin tone
└── AvatarHair — hair shape; Path driven by hair style + gender
```

```
const SKIN_TONES = ['#FDDDB4', '#D4956A', '#A0522D', '#4A2912'];

const BODY_WIDTHS = [28, 36, 40, 48]; // shoulder width in SVG units per variant
// Male body: rectangular torso; Female body: slightly tapered torso

const HAIR_PATHS = {
  male: {
    0: 'M...', // short — close-cropped cap shape
    1: 'M...', // long — hair falling to shoulders
    2: 'M...', // curly — rounded irregular shape
    3: '',     // shaved — no path, bare scalp
  },
  female: {
    0: 'M...', // short bob
    1: 'M...', // long straight
    2: 'M...', // curly volume
    3: 'M...', // bun / updo
  },
};
```

Positioning in the kitchen scene:

The avatar stands left-of-centre in front of the table. Its feet sit at the table surface line ($y \approx 178$ in the 400×300 coordinate grid), making it look like a person standing behind the table. The avatar height is approximately 110 SVG units — tall enough to be prominent but not overwhelming the kitchen.

```
// Inside KitchenScene — avatar is the top layer
<KitchenAvatar
  x={130}           // left-of-centre horizontal position
  gender={config.avatarGender}
  width={config.avatarWidth}
  skinTone={config.avatarSkinTone}
  hairStyle={config.avatarHairStyle}
/>
```

Avatar customiser UI (`AvatarCustomiser` component):

Used inside both `CreateProfileScreen` and `EditProfileScreen`. Shows a live preview of the avatar above four selection rows:

```
| [live avatar preview] |
|
| Gender:  [Male ●] [Female ○] |
| Width:   [○] [●] [○] [○]   |
| Skin tone: [👤] [👤] [👤] [👤] |
| Hair style: [○] [●] [○] [○] |
```

- Gender: two-option toggle (`male` / `female`)
- Width: row of 4 silhouette icons (simple body outline at each width)
- Skin tone: row of 4 filled circles in the skin tone colours
- Hair style: row of 4 small head-and-hair icons

Selecting any option updates a local state object and re-renders the preview avatar in real time. On save, the four values are written to the profile row.

Verification:

- Create male profile, slim, light skin, short hair → avatar appears in kitchen scene at correct position
- Create female profile, larger width, dark skin, long hair → distinct avatar visible
- Switch active profile → kitchen scene avatar changes immediately
- Collaborative view mini-kitchen scenes each show the correct avatar for that profile

Key files: `src/components/kitchen/KitchenAvatar.tsx`, `src/components/AvatarCustomiser.tsx`

R10 — iOS Only

- `app.json`: `"platforms": ["ios"]`
- All packages above have iOS support; none require Android-specific native modules beyond what Expo manages
- `expo-secure-store` maps to iOS Keychain natively
- For demo (Expo Go), no native build is needed at all — Expo Go itself provides the native layer

Full Package List

```
{
  "dependencies": {
    "expo": "~51.0.0",
    "expo-router": "~3.5.0",
    "expo-sqlite": "~14.0.0",
    "expo-secure-store": "~13.0.0",
    "expo-status-bar": "~1.12.0",
    "react": "18.2.0",
    "react-native": "0.74.0",
    "react-native-reanimated": "~3.10.0",
    "react-native-gesture-handler": "~2.16.0",
    "react-native-safe-area-context": "4.10.0",
    "react-native-screens": "3.31.0",
    "react-native-svg": "15.2.0",
    "zustand": "^4.5.0",
    "drizzle-orm": "^0.30.0",
    "@react-native-async-storage/async-storage": "1.23.1",
    "crypto-js": "^4.2.0",
    "uuid": "^9.0.0",
    "victory-native": "^41.0.0",
    "react-native-modal-datetime-picker": "17.1.0",
    "@gorhom/bottom-sheet": "^4.6.0",
    "date-fns": "^3.6.0"
  },
  "devDependencies": {
    "drizzle-kit": "^0.20.0",
    "typescript": "~5.3.0",
    "@types/crypto-js": "^4.2.2",
    "jest": "^29.0.0",
    "@testing-library/react-native": "12.0.0"
  }
}
```

Navigation Map

```
Expo Router (file-based)
├── (auth)/
│   ├── _layout.tsx    Stack navigator; shown when auth.store.userId === null
│   ├── login.tsx
│   └── create-account.tsx
└── (tabs)/
    ├── _layout.tsx    Tab navigator + profile switcher in header
    ├── index.tsx      Home (Kitchen scene) → /customise (pushed modal)
    ├── measurements.tsx  Top-tab: Timeline | Graph → /add (pushed modal)
    ├── profiles.tsx   Profile list → /create | /id/edit (pushed modals)
    └── collaborative.tsx All profiles view
```

Root `app/_layout.tsx` handles the auth redirect: if `userId === null` replace to `/(auth)/login`.

Implementation Sequence (Suggested Sprint)

Day	Work
1–2	Scaffold with <code>create-expo-app</code> ; install all deps; Drizzle schema + migrations; Auth service + Login/Create screens; Expo Router auth redirect; verify Expo Go on device
3–4	Profile CRUD screens; Zustand <code>activeProfile</code> store + profile switcher in tab layout; <code>AddMeasurementScreen</code> ; <code>TimelineScreen</code> <code>FlatList</code>

Day	Work
5	SVG assets (or coloured-container placeholders); <code>KitchenScene</code> component; <code>KitchenAvatar</code> component; <code>AvatarCustomiser</code> ; Home screen; <code>CustomiseKitchenScreen</code>
6	<code>GraphScreen</code> with Victory Native; milestone toggle in add form; milestone filter in timeline; milestone markers on graph
7	<code>CollaborativeScreen</code> ; full manual verification checklist; <code>eas build</code> or Expo Go QR for demo

Verification Checklist (Pre-Demo)

Auth: First launch → Create Account; force-quit restart → Login (not Create Account); correct/wrong password behaviour.

Profiles: Create 2 profiles; independent data; delete removes measurements.

Kitchen Scene: All 4 elements compose; customise updates immediately; Profile A and B hold separate configs; switching active profile changes scene.

Measurements: Add today's date + historical date; timeline correct order; graph plots both as connected line. Test all three units: enter 72.5 kg → displays "72.5 kg"; enter 159.8 lbs → displays "159.8 lbs"; enter 11 st 4 lbs → displays "11 st 4 lbs". Verify graph y-axis unit label matches profile setting. Verify stones profile shows lbs on y-axis with stones/lbs in tooltip.

Milestones: Add milestone with name; badge appears in timeline; milestone filter shows only that row; amber marker on graph.

Profile Switching: Switch → kitchen scene + measurements + graph all update; restart → last profile restored.

Collaborative: All cards visible; correct latest weight + milestone label; adding measurement updates card live.

Expo Go / Device: Scan QR code from `npx expo start` → opens instantly; no "Trust developer" dialog needed (Expo Go is already trusted).

Key Risks

Risk	Mitigation
SVG artwork not ready in time	Use <code>View</code> with <code>backgroundColor</code> per layer; component API unchanged when SVGs arrive
Victory Native chart not rendering on first try	Check peer dep on <code>react-native-reanimated</code> ; ensure <code>reanimated</code> babel plugin in <code>babel.config.js</code>
Drizzle migration conflicts	For demo, run <code>drizzle-kit push --force</code> to reset; no prod data to preserve
Expo SDK version conflicts between packages	Use <code>npx expo install <pkg></code> (not <code>npm install</code>) — Expo CLI auto-resolves compatible versions
Bottom sheet / gesture handler setup	Follow @gorhom/bottom-sheet Expo setup docs exactly; wrap root in <code>GestureHandlerRootView</code>